

音乐数据爬取与展示系统

实验报告

数据采集 · 数据库管理 · Web 展示 · 文本分析

摘要

本项目以酷我音乐公开页面和接口返回的数据为对象，完成了歌手与歌曲信息的采集、歌词抓取、SQLite 数据库存储、Django Web 展示以及歌词和歌曲标题的文本分析。系统能够提供歌曲列表、歌手列表、歌曲详情、歌手详情和歌曲/歌手搜索等功能；分析部分完成了歌词清洗、中文分词、停用词处理、词频统计和歌手平均歌词长度比较。

关键词：网络爬虫；Django；SQLite；中文分词；词频分析；歌词长度

一、项目背景与实验目标

音乐网站中的歌手、歌曲和歌词具有较强的结构化与文本化特征，适合用于练习网络数据采集、关系数据库设计、Web 应用开发和自然语言处理。本实验的目标是构建一个从数据采集到数据展示、再到文本分析的完整小型系统。

掌握基于 requests 和 BeautifulSoup 的网页/API 数据采集方法。

完成歌手、歌曲、封面、链接、简介和歌词等多源字段的整合。

建立 Artist-Song 一对多关系，并使用 Django ORM 访问数据。

实现歌词清洗、中文分词、停用词去除和基础统计分析。

通过图表和报告展示数据分析结果，并说明数据局限性。

二、系统总体设计

系统采用“数据采集层—数据处理层—数据存储层—Web 展示层—分析输出层”的结构。采集脚本负责从网站获取原始数据；数据处理脚本负责清洗和分词；Django 管理命令将数据导入 SQLite；视图函数和模板负责页面呈现；分析脚本输出 CSV、SVG/PNG 图表和报告文档。

层次	主要职责	对应文件/技术
数据采集层	获取歌手列表、歌曲信息、歌词和链接	requests、BeautifulSoup、JSON API
数据处理层	歌词解析、清洗、分词、停用词处理	Python、正则表达式、jieba
数据存储层	关系化保存歌手与歌曲数据	Django ORM、SQLite、JSONField
Web 展示层	列表、详情、分页和搜索	Django Views、URL、HTML/CSS 模板
分析输出层	词频、平均长度、图表和报告	Counter、SVG、ReportLab

三、数据采集与数据量

项目通过多个脚本分阶段采集数据。首先从歌手列表页面提取歌手名称、头像和 `artist_id`；随后调用歌手歌曲接口获取歌曲名称、`song_id` 和封面；再调用歌词接口获取逐行歌词；最后补充歌手简介、歌手链接和歌曲详情链接。请求过程中使用 `User-Agent`、`Referer`、请求间隔和超时设置，以降低连续请求带来的风险。

数据对象	数量/范围	主要字段
歌手原始数据	119 位	<code>name</code> 、 <code>artist_id</code> 、 <code>img_url</code> 、 <code>info</code> 、 <code>url</code>
歌曲原始记录	2367 条	<code>artist_id</code> 、 <code>artist_name</code> 、 <code>song_name</code> 、 <code>song_id</code> 、 <code>song_photo</code> 、 <code>lyric</code>
去重后歌曲	2318 首	以 <code>song_id</code> 作为唯一标识
数据库歌手	119	<code>songs_artist</code> 表
数据库歌曲	2318	<code>songs_song</code> 表
单歌手收录数量	14—20 首	受爬取接口和 <code>rn</code> 参数影响

数据质量情况

原始歌曲记录与唯一 `song_id` 数量存在差异，说明有少量重复记录。数据库通过 `song_id` 的唯一约束和 `update_or_create` 方式进行整合。歌词字段中部分记录为空或属于纯音乐、电影原声等无正文内容，因此在文本分析前进行了单独识别和排除。

四、数据库设计与数据导入

系统使用 Django 默认项目结构，并在 songs 应用中设计 Artist 和 Song 两个核心模型。Artist 保存歌手基本信息；Song 保存歌曲基本信息、歌词 JSON 和所属歌手。Song.artist 使用外键关联 Artist，形成一名歌手对应多首歌曲的一对多关系。

模型	核心字段	设计说明
Artist	artist_id、name、img_url、info、url	artist_id 唯一；保存歌手资料和原始链接
Song	song_id、name、artist_name、photo_url、lyric、artist、url	song_id 唯一；lyric 使用 JSONField 保存逐行歌词

数据导入使用自定义 Django management command ``import_music_data``，采用事务保证批量导入的一致性。导入时先根据 `artist_id` 创建或更新歌手，再根据歌曲的 `artist_id` 找到外键对象，最后创建或更新歌曲记录。

五、系统主要功能

功能	访问路径	实现效果
歌曲列表	/	分页展示歌曲、封面、歌手和原始链接
歌曲详情	/songs//	展示封面、歌名、歌手和逐行歌词
歌手列表	/artists/	分页展示歌手、头像和收录歌曲数量
歌手详情	/artists//	展示简介、头像、原始链接和歌曲列表
搜索	/search/?q=...&type;=...	按歌曲名或歌手名模糊搜索并显示耗时
后台管理	/admin/	使用 Django Admin 管理模型数据

六、数据清洗与分析算法

歌词原始值不是普通文本，而是字符串形式的列表，每个元素通常包含 lineLyric 和 time。清洗程序首先使用 ast.literal_eval 或 JSON 解析列表，再提取歌词行。随后删除歌名标题、作词/作曲/编曲等制作信息、时间戳、HTML 标签、括号提示、纯音乐说明和多余符号，最终将清洗文本保存为 lyric_clean，同时保留 lyric_raw 以便追溯。

中文分词与停用词

使用 jieba 对 lyric_clean 进行中文分词，并同时保留完整分词结果和去除停用词后的结果。停用词主要包括常见代词、虚词、语气词以及 Do、Re、Mi 等唱名。完整结果用于追溯，lyric_tokens_no_stopwords 用于词频分析。

主要统计方法

- 总词频：使用 Counter 汇总所有有效歌词中的词语出现次数。
- 歌曲标题词频：先清理 DJ 版、Live、Version、feat 等版本标记，再按 song_id 去重后分词统计。
- 平均歌词长度：统计每首有效歌词中中文、英文和数字字符数，再按歌手计算平均值。
- 异常控制：空歌词、纯音乐、无正文提示和解析失败记录不进入歌词长度统计。

七、分析结果展示

以下图表来自本项目实际运行结果，分别展示歌词词频、歌曲标题词频和歌手平均歌词长度。

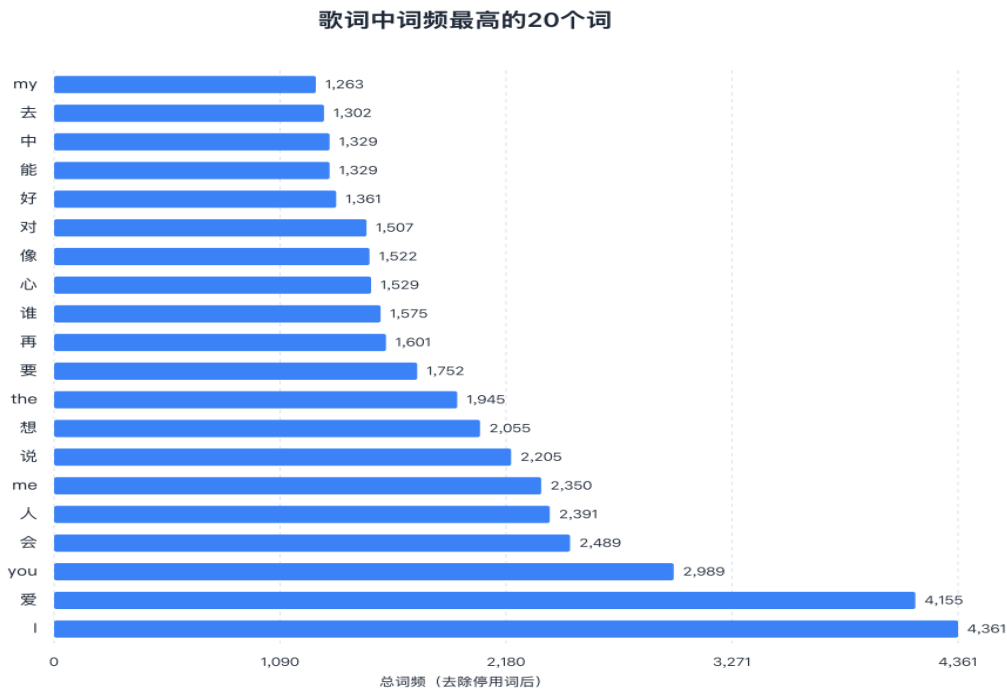


图 1 歌词中词频最高的 20 个词

歌词词频结果显示，‘爱’是最突出的中文主题词；I、you、me 等人称词频较高，说明歌词常采用个人感受和人物关系的叙述方式。

歌曲标题中词频最高的20个词

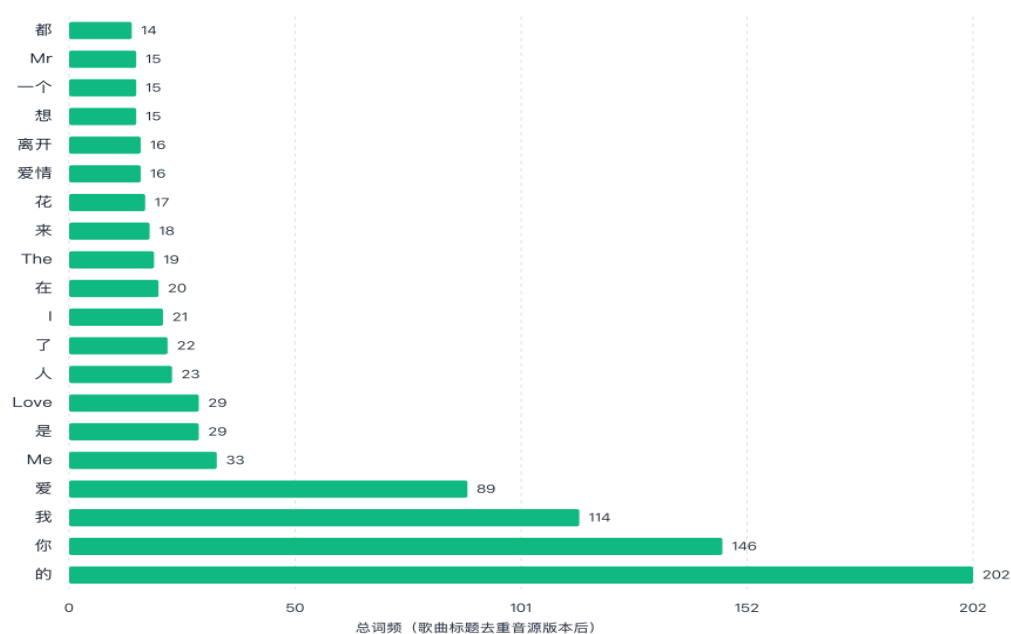


图2 歌曲标题中词频最高的20个词

歌曲标题中的‘你’‘我’‘爱’‘爱情’等词排名靠前，说明爱情和人物关系是歌曲命名的重要主题。标题相较歌词正文更简洁，倾向于使用少量概括性情感词或意象词。

歌手平均歌词长度 Top 10

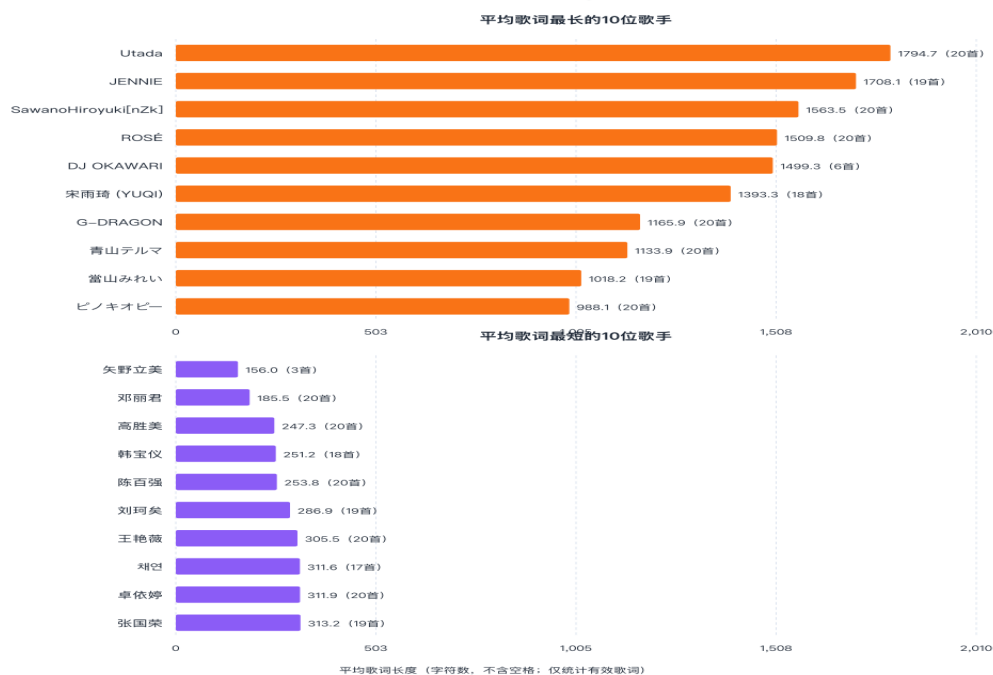


图3 歌手平均歌词长度最长与最短的10位歌手

不同歌手的平均歌词长度差异明显，反映了文本规模、叙事程度和歌曲结构的差异。平均值较短的歌手手中可能包含配乐或歌词较少的作品，因此‘短’不一定等同于创作风格简单。

八、实验过程与结果评价

本项目完成了从网络数据获取、结构化存储到 Web 展示和文本分析的完整流程。数据库中歌手和歌曲记录能够通过外键关联；列表页面支持分页，详情页面支持关联浏览，搜索页面支持按类型查询；分析流程能够输出可复用的中间数据和图表文件。

系统优点

采用分阶段脚本，采集流程清晰，便于单独重跑某一阶段。

数据库模型简单明确，Artist-Song 关系符合实际业务结构。

搜索、分页、详情和原始链接跳转构成了完整的基础音乐数据展示功能。

分析中保留原始歌词和清洗字段，具有一定的数据可追溯性。

系统局限

歌手歌曲数量主要受接口参数和爬取规则影响，不能代表完整作品数量。

当前没有播放量、收藏量、评分、发行时间和曲风等字段，不能直接分析热度、年代趋势或受欢迎程度。

中英文停用词处理仍需统一，英文人称词与中文人称词的统计口径存在差异。

部分歌手有效歌词数量较少，平均歌词长度的排名稳定性有限。

项目使用 `DEBUG=True` 和开发环境配置，正式部署时需要完善安全配置、异常处理和访问限制。

九、总结

本实验构建了一个具有数据采集、数据库管理、Web 展示和文本分析能力的音乐数据系统。通过歌词和歌名分析，可以观察当前样本中的爱情、人际关系和个人情绪表达；通过平均歌词长度，可以比较不同歌手在文本规模和表达结构上的差异。后续若补充发行时间、歌曲类型和平台热度字段，还可以继续开展年代趋势、曲风比较和热度预测等研究。